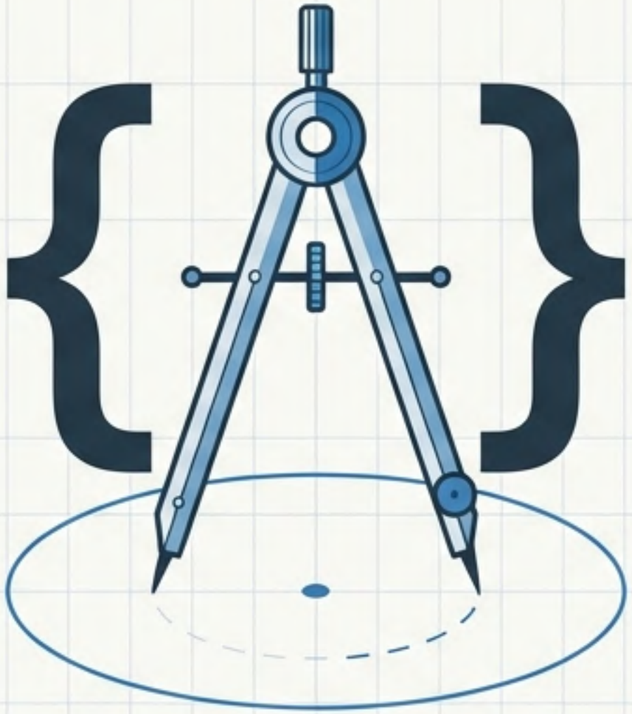
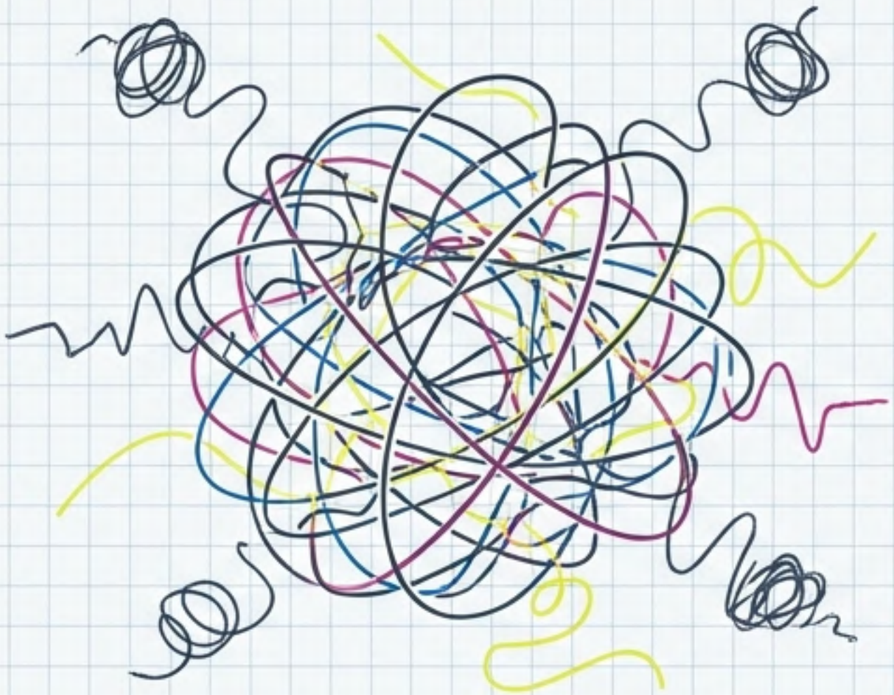


El Blueprint del Frontend: Guía de Diseño y Planificación Web

De la idea inicial a una arquitectura escalable en 4 pasos.

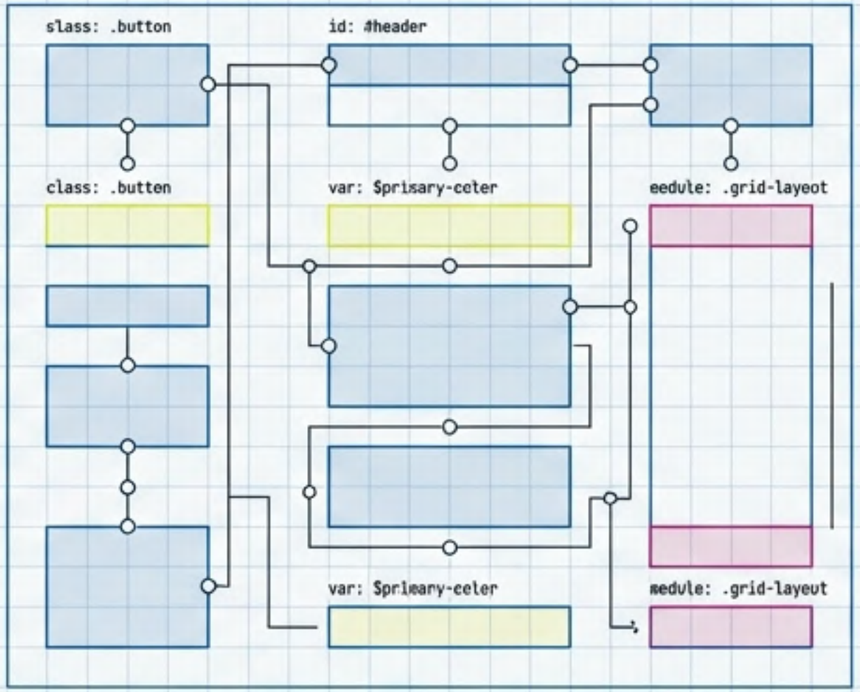
NotebookLM

El Caos



Sin planificación, el CSS crece de forma desordenada. Se vuelve impredecible, difícil de mantener y genera miedo al modificarlo.

El Blueprint

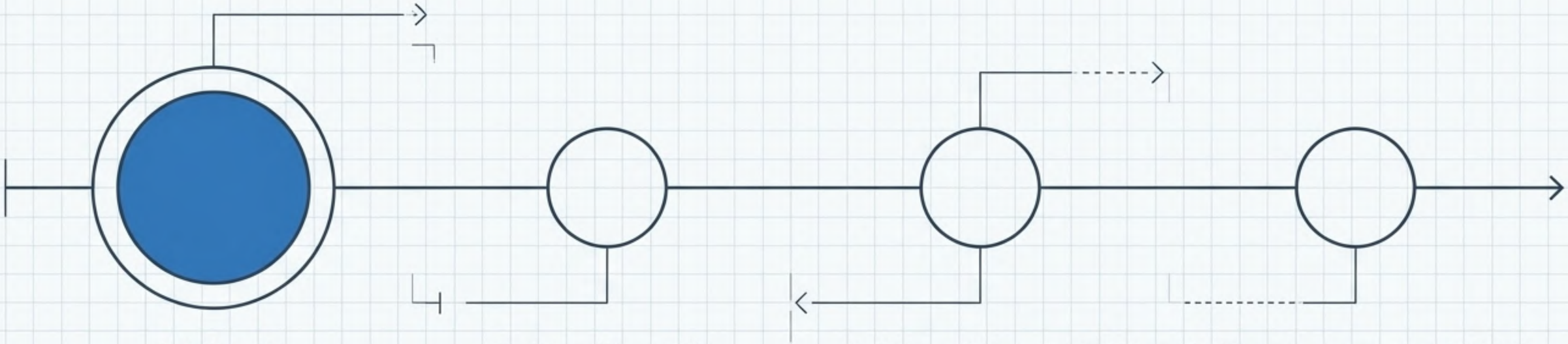


Con arquitectura, el código es predecible. Cada clase tiene un propósito y cada archivo tiene su lugar.

Escribir buen CSS es 80% planificación y 20% código.

NotebookLM

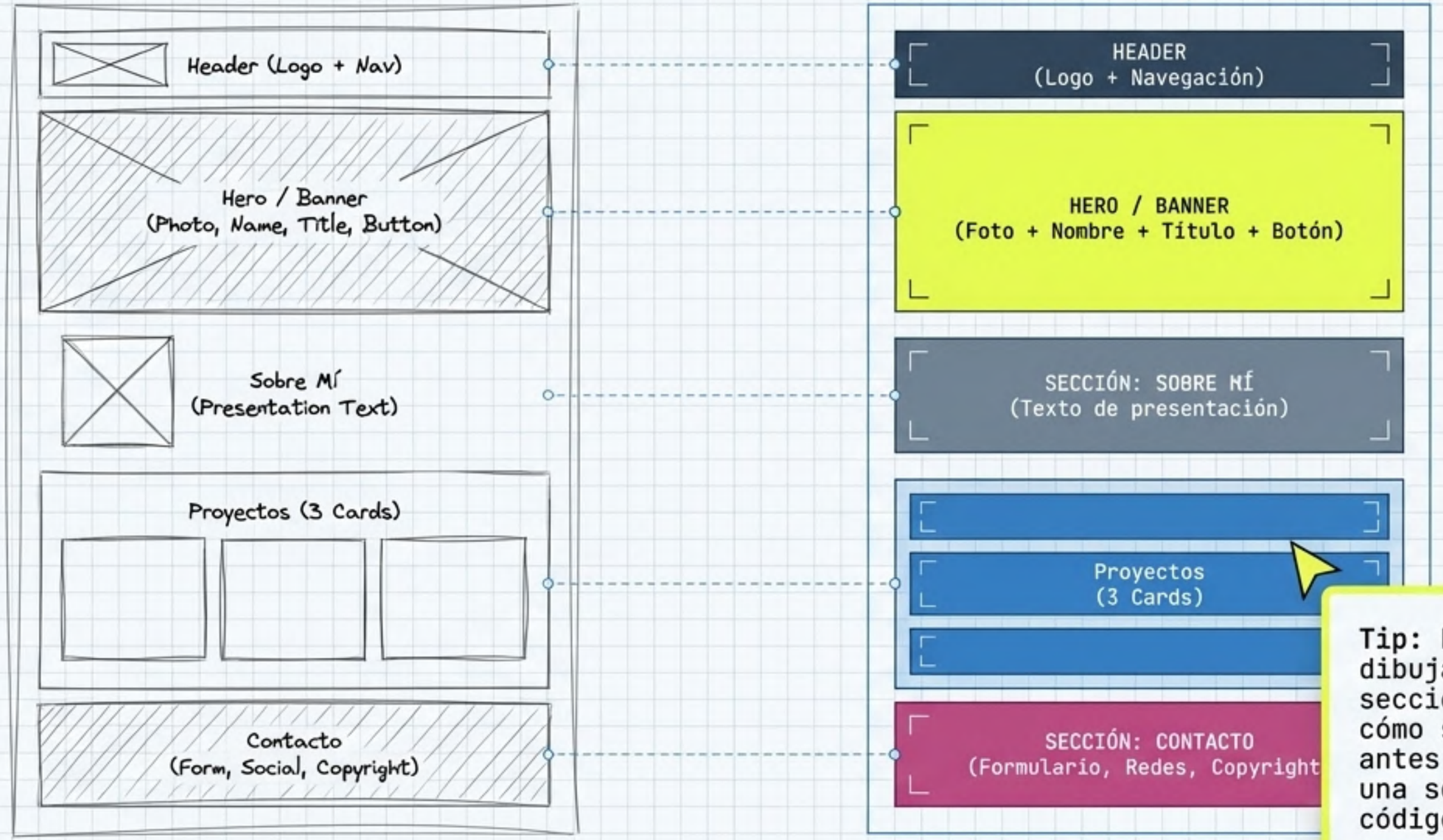
El Mapa de Ruta: 4 Pilares del Proyecto Web



1. Diseño (Wireframes y bloques visuales)
2. Planificación (Estructura de archivos)
3. Metodología CSS (Reglas de nomenclatura)
4. Preprocesador SASS (Herramientas avanzadas)

NotebookLM

Pilar 1: El Diseño (Maquetación)

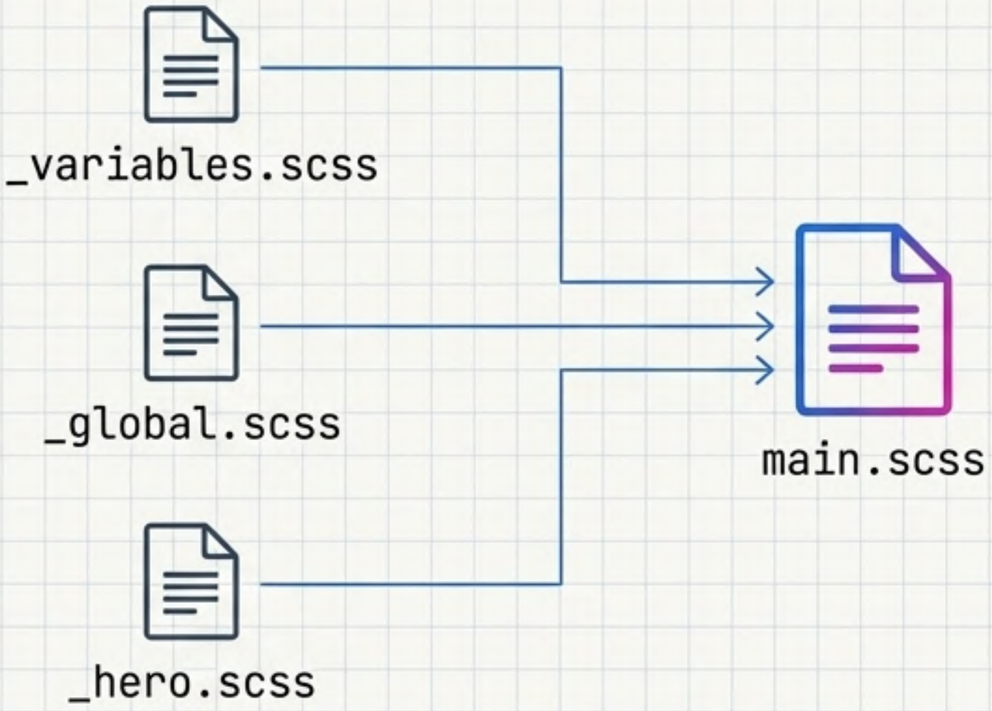


Tip: El paso cero es dibujar. Define qué secciones tendrás y cómo se organizan antes de escribir una sola linea de código.

NotebookLM

Pilar 2: Arquitectura del Proyecto

```
mi-portafolio/
    assets/
    scss/
        _variables.scss
        _global.scss
        _hero.scss
        main.scss
```



Divide y vencerás. Separar los archivos SCSS por sección hace que el código sea fácil de encontrar y mantener.

NotebookLM

Pilar 3: El Caos vs. El Orden (Metodologías)

Sin Metodología

```
.card { ... }
 .title { ... }
 .text { ... }
```

¿Este .title es del header, del hero, o de la card? (Conflicto de nombres).

Con BEM

```
.card { ... }
.card__title { ... }
.card__text { ... }
```

Queda claro: este título pertenece a la card.

NotebookLM

Matriz de Decisión: Metodologías CSS

Metodología	Idea Principal	Ejemplo de clase
BEM	Bloque__Elemento--Modificador	Recomendado `.card__title--large`
SMACSS	Categoriza estilos en 5 tipos	`.is-hidden, .btn-action` JetBrains Mono
OOCSS	Separa estructura de apariencia	`.box, .bg-primary` JetBrains Mono

¿Por qué elegimos BEM? Es la más fácil de aprender, el nombre de la clase dice a qué componente pertenece, y evita conflictos al ser única y descriptiva.

NotebookLM

La Anatomía de BEM

`.hero__button--primary`

Bloque: El componente principal independiente.

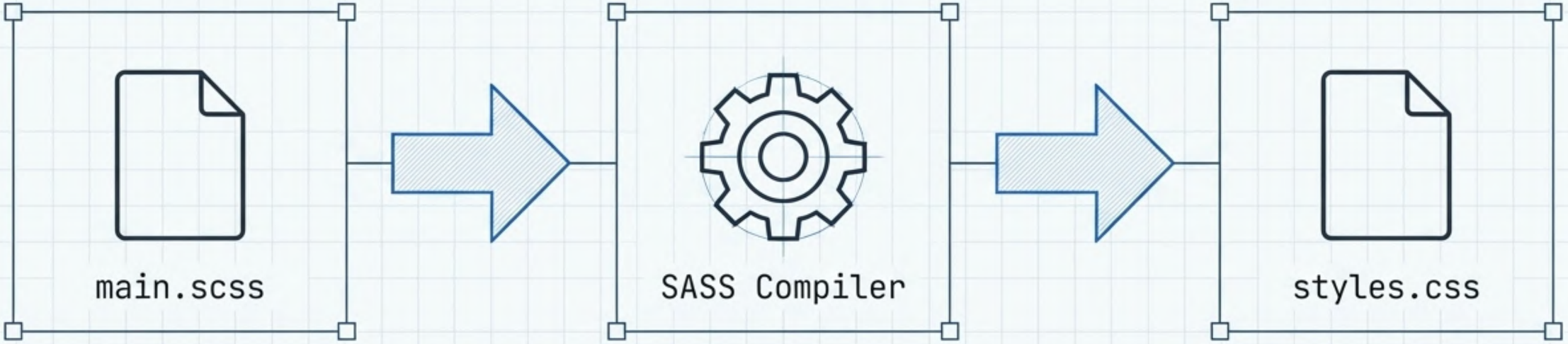
Elemento: Una parte del bloque que no tiene sentido por sí sola. Usa doble guion bajo __.

Modificador: Una variación de apariencia o estado. Usa doble guion medio --.

```
<div class="hero">
  <h2 class="hero__title">…</h2>
  <button class="hero__button hero__button--primary">...</button>
</div>
```

NotebookLM

Pilar 4: ¿Qué es un Preprocesador?



Código con "superpoderes" (variables, anidamiento, funciones).

El Motor (Traduce el código).

CSS normal que el navegador entiende.

Es una herramienta que te permite escribir CSS con **lógica de programación**, y luego lo convierte a **CSS estándar** de forma **automática.**

NotebookLM

Selección de Herramientas: Preprocesadores

Preprocesador	Extensión	Característica
SASS/SCSS	.scss	El más popular, sintaxis parecida a CSS. Nuestra elección
LESS	.less	Simple, usado por Bootstrap 3.
Stylus	.styl	Muy flexible, sintaxis minimalista.

¿Por qué SASS?

- Preprocesador más usado en la industria.
- La sintaxis SCSS es casi idéntica a CSS (curva de aprendizaje baja).
- Permite variables, anidamiento y archivos parciales.
- Gran comunidad y documentación.

NotebookLM

La Pareja Perfecta: SASS + BEM

CSS Normal

```
.hero { … }
.hero__title { … }
.hero__button { … }
.hero__button--primary { … }
```

Se repite la palabra "hero" constantemente (Verboso).

SCSS (Anidamiento)

```
.hero {
  &__title { … }
  &__button {
    &--primary { … }
  }
}
```

El símbolo & concatena el bloque padre. Hace que escribir código BEM sea exponencialmente más rápida y organizado.

NotebookLM

El Blueprint Completado

1. Diseño
(Wireframes definidos)

2. Planificación
(Estructura de carpetas)

3. Metodología
(Nomenclatura BEM)

4. Preprocesador
(SASS/SCSS configurado)

🚀 Ahora es tu turno.

Elige un tema para tu página web. Dibuja tu wireframe, crea tu estructura de carpetas scss, elige BEM, y compila tu código. ¡Construye con arquitectura!

NotebookLM